DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: **DATABASE MANAGEMENT SYSTEMS** Subject Code: **CS T53**

UNIT –IV

Query Processing: Measures of Query Cost – Selection Operation – Sorting – Join Operation – Other Operations – Evaluation of Expressions

Query Optimization – Overview – Transformation of Relational Expressions – Estimating Statistics of Expression Results – Choice of Evaluation Plan

Transactions – Concept – A Simple Transaction Model – Storage Structure – Transaction Atomicity and Durability – Transaction Isolation – Serializability – Transaction Isolation and Atomicity – Transaction Isolation Levels – Implementation of Isolation Levels – Transactions as SQL Statements

TWO MARK

1. What is transaction? (NOV 2014)(NOV 2015)

Transaction is a unit of a program execution that accesses and possibly updates various data items

2. What are the types of transaction?

a. begin transaction

b. end transaction

3. What are the properties of transaction(OR)Explain ACID?(April 2018)

- Atomicity
- Consistency
- Isolation
- Durability

4. What is atomicity?

Either all the operation of the transaction is reflected properly in the database or none are reflected properly in the database.

5. What is consistency?

Execution of the transaction in isolation preserves the consistency of the database.

6. What is isolation?

Even though multiple transactions may execute concurrently. The system guarantees every pair of transaction Ti and Tj.. it appears to Ti that either Tj finished the execution before Ti started. Thus each transaction is unaware of other transaction executing concurrently in the system.

7. What is durability?

After the transaction completes successfully the change it has made to the database persist even if there are system failures.

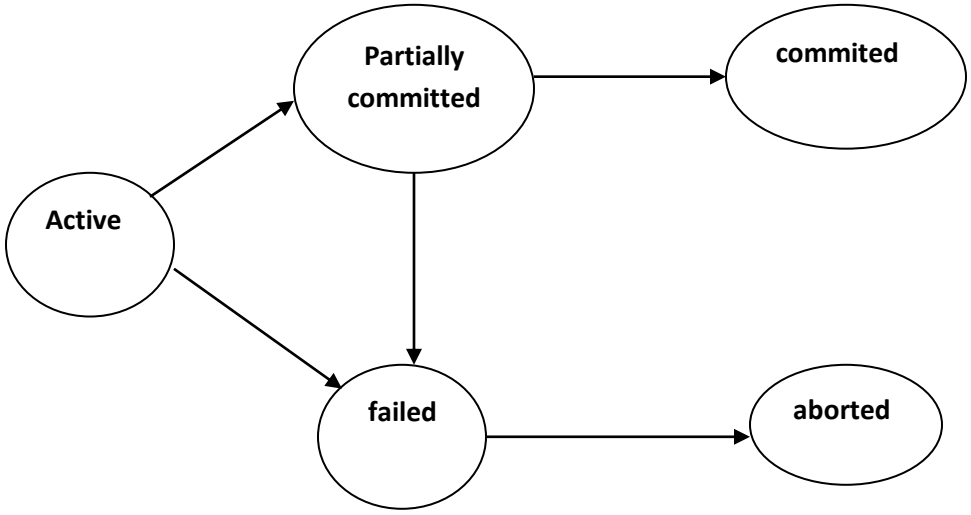
8. What are the operations of the transaction?

- Read(x) –which transfers the data items x from the database to the local buffer belonging to the transaction that executed the read operation.
- Write(x)- which transfers the data items x from the local buffer of the transaction that executed the write back to the database.

9. What is the transaction state?

- Active
- Partially committed
- Failure
- Aborted
- Committed

10. Draw the state diagram of the transaction? (APR 2014)



11. What is reduce waiting time?

Concurrent execution reduces unpredictable delays in a running transaction. Moreover it is also reduces the average response time. The average time for the transaction to be completed after it has been submitted.

12. What is serializability? (NOV 2015) **(DEC2018)(MAY 2019)**

The database system must control concurrent execution of the transaction to ensure that the database state remains consistent.

13. What are the two types of serializability?

- Conflict serializability
- View serializability

14. What is conflict serializability?

- Ii =read(q), Ij=read(q)
- Ii= read(q), Ij=write(q)
- Ii=write(q), ij=read(q)
- Ii=write(q),Ij=write(q)

15. What is view serializability?

- For each data item Q if transaction Ti reads the initial value of q in schedule S. then the transaction Ti must in schedule S' also reads the initial value Q.
- For each data item Q if the transaction Ti executes read(Q) in schedule S and if that value was produced by the write(Q) operation executed by the transaction Tj.
- For each data item Q the transaction that performs the final write(Q) operation in schedule S must perform the final write((Q) operation in schedule S'.

16. What is recoverability?

In a system that allows concurrent execution, it is necessary also to ensure that any transaction Tj that is dependent on Ti is also aborted. To achieve this we need to place restriction on the type of schedules permitted in the system.

17. What is cascading rollback?

The phenomenon in which single transaction failure leads to the series of transaction rollback is called cascading rollback.

18. What is cascadeless schedule?

Cascading rollback is undesirable since it leads to the undoing of the significant amount of work. It is desirable to restrict the schedule to those where cascading rollbacks cannot occur. Such schedule are called cascadless schedule.

19. What is query processing?(Nov 2017) **(MAY 2019)**

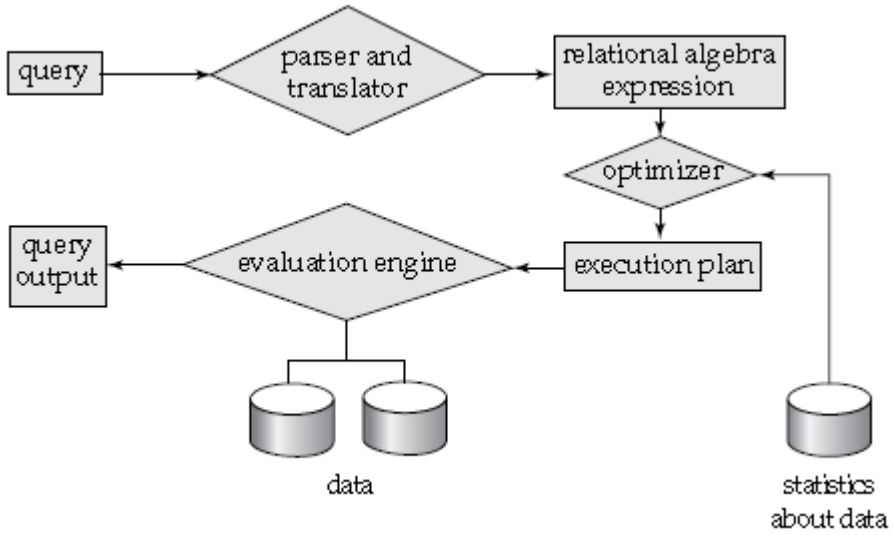
Query processing refers to the range of activities involved in extracting data from a database. The activities include translation of queries in high-level database languages into expressions that can be used at the physical level of the file system, a variety of query-optimizing transformations, and actual evaluation of queries.

20. Basic steps in query processing.(Nov 2017)

The steps involved in processing a query are

1. Parsing and translation
2. Optimization
3. Evaluation

21. Describe the diagrammatic representation of query processing.

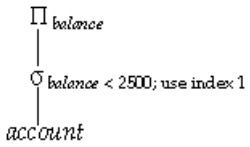


22. Explain the terms annotations and query evaluation primitive.

Annotations may state the algorithm to be used for a specific operation, or the particular index or indices to use. A relational-algebra operation annotated with instructions on how to evaluate it is called an **evaluation primitive**.

23. What is query execution plan and query execution engine(April 2018)

A sequence of primitive operations that can be used to evaluate a query is a **query execution plan** or **query-evaluation plan**. Figure illustrates an evaluation plan in which a particular index is specified for the selection operation. The **query-execution engine** takes a query-evaluation plan, executes that plan, and returns the answers to the query.



24. What are the various measures of query cost.

The cost of query evaluation can be measured in terms of a number of different resources, including

i. disk accesses ,
ii. CPU time to execute a query
iii. the cost of communication
iv. the response time for a query-evaluation plan.

25. Explain the terms used in block transfer.

Rotational latency: waiting for the desired data to spin under the read–write head

Seek Time: the time that it takes to move the head over the desired track or cylinder

Sequential I/O: where the blocks read are contiguous on disk

Random I/O: where the blocks are noncontiguous

26. What are the basic algorithms in selection operation

- Linear Search A1 algorithm.
- Binary Search A2 algorithm

27. What is Linear search.

In a linear search, the system scans each file block and tests all records to see whether they satisfy the selection condition. For a selection on a key attribute, the system can terminate the scan if the required record is found, without looking at the other records of the relation.

28. What is Binary search.

If the file is ordered on an attribute, and the selection condition is an equality comparison on the attribute, we can use a binary search to locate records that satisfy the selection. The system performs the binary search on the blocks of the file.

29. Explain selection using index structures.

Index structures are referred to as **access paths.** Search algorithms that use an index are referred to as **index scans**.

- **A3** (**primary index, equality on key**). For an equality comparison on a key attribute with a primary index, we can use the index to retrieve a single record that satisfies the corresponding equality condition.
- **A4** (**primary index, equality on nonkey**). We can retrieve multiple records by using a primary index when the selection condition specifies an equality comparison on a nonkey

attribute, *A*. The only difference from the previous case is that multiple records may need to be fetched.

- **A5** (**secondary index, equality**). Selections specifying an equality condition can use a secondary index. This strategy can retrieve a single record if the equality condition is on a key; multiple records may get retrieved if the indexing field is not a key.

30. Explain the implementation of complex queries.

- **Conjunction:** A *conjunctive selection* is a selection of the form
  - $\sigma\theta 1 \wedge \theta 2 \wedge \cdots \wedge \theta n(r)$
- **Disjunction:** A *disjunctive selection* is a selection of the form
  - $\sigma\theta 1 \vee \theta 2 \vee \cdots \vee \theta n(r)$
- **Negation:** The result of a selection $\sigma \neg\theta(r)$ is the set of tuples of r for which the condition θ evaluates to false.

31. What is External sorting. (Nov 2014)

Sorting of relations that do not fit in memory is called **external sorting**. The most commonly used technique for external sorting is the **external sort–merge** algorithm. Let *M* denote the number of page frames in the main-memory buffer (the number of disk blocks whose contents can be buffered in main memory).

- In the first stage, a number of sorted **runs** are created; each run is sorted, but contains only some of the records of the relation.
- In the second stage, the runs are *merged*. Suppose, for now, that the total number of runs, *N*, is less than *M*, so that we can allocate one page frame to each run and have space left to hold one page of output.

32. What is block nested join.(DEC 2018)

Block nested-loop join is a variant of the nested-loop join where every block of the inner relation is paired with every block of the outer relation. Within each pair of blocks, every tuple in one block is paired with every tuple in the other block, to generate all pairs of tuples. As before, all pairs of tuples that satisfy the join condition are added to the result.

33. What are the types of join operation used.

- Nested loop join
- Block nested loop join
- Indexed nested loop join
- Merge join

- Hash join

34. What is hash table overflow?

Hash-table overflow occurs in partition *i* of the build relation *s* if the hash index on *Hsi* is larger than main memory. Hash-table overflow can occur if there are many tuples in the build relation with the same values for the join attributes, or if the hash function does not have the properties of randomness and uniformity

35. What is Fudge factor?

We can handle a small amount of skew by increasing the number of partitions so that the expected size of each partition (including the hash index on the partition) is somewhat less than the size of memory. The number of partitions is therefore increased by a small value called the **fudge factor.**

36. What is overflow resolution?

Overflow resolution is performed during the build phase, if a hash-index overflow is detected. Overflow resolution proceeds in this way: If H_{si} , for any *i*, is found to be too large, it is further partitioned into smaller partitions by using a different hash function. Similarly, H_{ri} is also partitioned using the new hash function, and only tuples in the matching partitions need to be joined.

37. What is overflow avoidance?

Overflow avoidance performs the partitioning carefully, so that overflows never occur during the build phase. In overflow avoidance, the build relation *s* is initially partitioned into many small partitions, and then some partitions are combined in such a way that each combined partition fits in memory. The probe relation *r* is partitioned in the same way as the combined partitions on *s*, but the sizes of H_{ri} do not matter.

38. What are the ways in which pipeline can be executed.

Pipelines can be executed in either of two ways:

- **Demand driven**: In a **demand-driven pipeline**, the system makes repeated requests for tuples from the operation at the top of the pipeline. Each time that an operation receives a request for tuples, it computes the next tuple (or tuples) to be returned, and then returns that tuple.
- **Producer driven**: operations do not wait for requests to produce tuples, but instead generate the tuples **eagerly**. Each operation at the bottom of a pipeline continually generates output tuples, and puts them in its output buffer, until the buffer is full.

What is query optimization.(April 2014) (April 2015) **[DEC 2016]**

39.

Query optimization is the process of selecting the most efficient query-evaluation plan from among the many strategies usually possible for processing a given query, especially if the query is complex.

40. How authorization provided to the users in SQL? (April 2011)

Developing an application using a least-privileged user account (LUA) approach is an important part of a defensive, in-depth strategy for countering security threats. The LUA approach ensures that users follow the principle of least privilege and always log on with limited user accounts. Administrative tasks are broken out using fixed server roles, and the use of the **sys admin** fixed server role is severely restricted.

41. What is an audit trail? (April 2011)

An audit trail is a log of changes (insert, delete, update) to the database along with information such as which user performed the change and when the change was performed.

42. List out the various storage devices (April/May 2012)

Storage Devices are the data storage devices that are used in the computers to store the data. The computer has many types of data storage devices. Some of them can be classified as the removable data Storage Devices and the others as the non removable data Storage Devices.

The memory is of **two types**; one is the **primary memory** and the other one is the **secondary memory**.

The primary memory is the volatile memory and the secondary memory is the non volatile memory. The volatile memory is the kind of the memory that is erasable and the non volatile memory is the one where in the contents cannot be erased. Basically when we talk about the data storage devices it is generally assumed to be the secondary memory. The secondary memory is used to store the data permanently in the computer. The secondary storage devices are usually as follows: hard disk drives – this is the most common type of storage device that is used in almost all the computer systems. The other ones include the floppy disk drives, the CD ROM, and the DVD ROM. The flash memory, the USB data card etc.

43. What are the steps in query processing? (April 2013)

- Parsing and Translation
- Optimization
- Evaluation

11 MARKS

1. Explain typically available storages media in detail (Apr 2011)

Classification of Physical Storage Media

- Speed with which data can be accessed
- Cost per unit of data
- Reliability
  - data loss on power failure or system crash
  - physical failure of the storage device
- Can differentiate storage into:
  - **volatile storage:** loses contents when power is switched off

Nonvolatile storage:

☐Contents persist even when power is switched off.

☐Includes secondary and tertiary storage, as well as

Battery backed up main memory

Physical Storage Media

Cache

Fastest and most costly form of storage; volatile; managed by the computer system hardware

(Note: "Cache" is pronounced as "cash")

Main memory

- Fast access (10s to 100s of nanoseconds; 1 nanosecond = 10–9 seconds)
- Generally too small (or too expensive) to store the entire database

  Capacities of up to a few Gigabytes widely used currently

  Capacities have gone up and per byte costs have decreased steadily and rapidly (roughly factor of 2 every 2 to 3 years)
- **Volatile** — contents of main memory are usually lost if a power

  Failure or system crash occurs.

Flash memory

- Data survives power failure
- Data can be written at a location only once, but location can be erased and written to again
    - Can support only a limited number (10K – 1M) of write/erase cycles.
    - Erasing of memory has to be done to an entire bank of memory
- Reads are roughly as fast as main memory
- But writes are slow (few microseconds), erase is slower
- NOR Flash
    - Fast reads, very slow erase, lower capacity
    - Used to store program code in many embedded devices
- NAND Flash
    - Page-at-a-time read/write, multi-page erase
    - High capacity (several GB)
    - Widely used as data storage mechanism in portable devices

Magnetic-disk

- Data is stored on spinning disk, and read/written magnetically
- Primary medium for the long-term storage of data; typically stores entire database.
- Data must be moved from disk to main memory for access, and written back for storage
- **direct-access** – possible to read data on disk in any order, unlike magnetic tape
- Survives power failures and system crashes
    - disk failure can destroy data: is rare but does happen

Optical storage

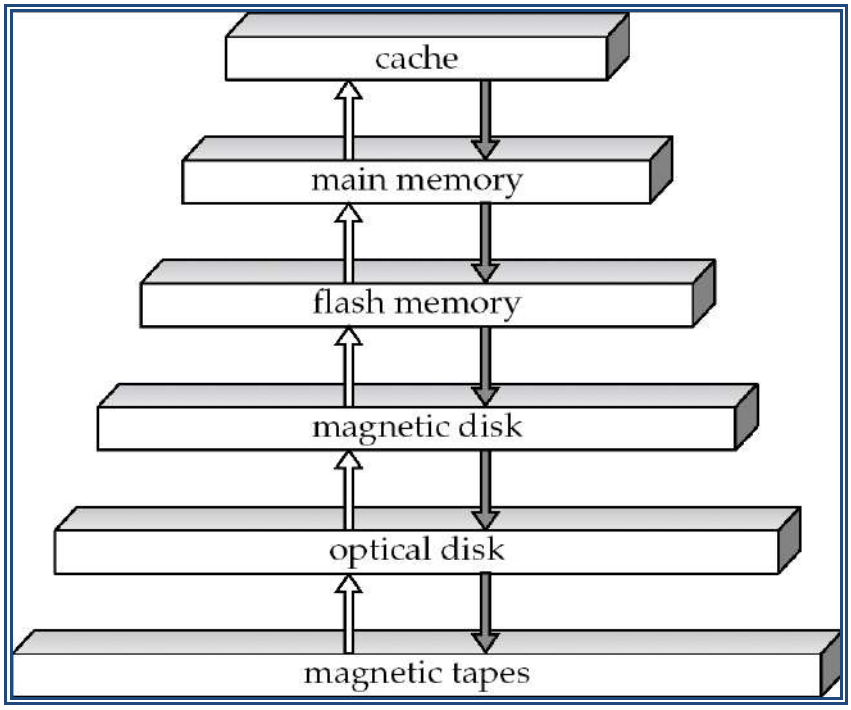
- non-volatile, data is read optically from a spinning disk using a laser
- CD-ROM (640 MB) and DVD (4.7 to 17 GB) most popular forms
- Write-one, read-many (WORM) optical disks used for archival storage (CD-R, DVD-R, DVD+R)
- Multiple write versions also available (CD-RW, DVD-RW, DVD+RW, and DVD-RAM)
- Reads and writes are slower than with magnetic disk

- **Juke-box** systems, with large numbers of removable disks, a few drives, and a mechanism for automatic loading/unloading of disks available for storing large volumes of data

Tape storage

- non-volatile, used primarily for backup (to recover from disk failure), and for archival data
- **sequential-access** – much slower than disk
- very high capacity (40 to 300 GB tapes available)
- tape can be removed from drive ⇒ storage costs much cheaper than disk, but drives are expensive
- Tape jukeboxes available for storing massive amounts of data
  - hundreds of terabytes (1 terabyte = 10^9 bytes) to even a petabyte (1 petabyte = 10^{12} bytes)

Storage Hierarchy

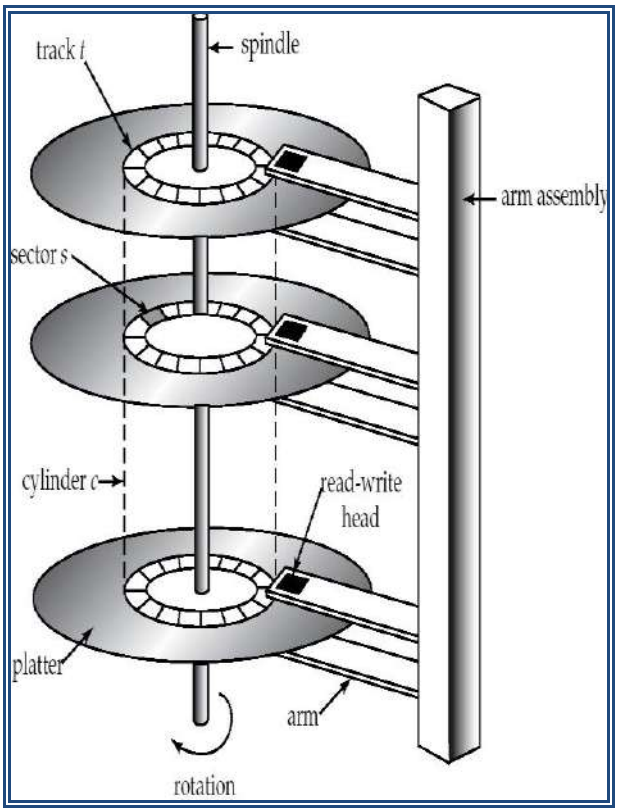


- primary storage: Fastest media but volatile (cache, main memory).
- secondary storage: next level in hierarchy, non-volatile, moderately fast access time

- ✓ also called on-line storage
- ✓ E.g. flash memory, magnetic disks

- tertiary storage: lowest level in hierarchy, non-volatile, slow access time
  - ✓ also called off-line storage
  - ✓ E.g. magnetic tape, optical storage

Magnetic Disk



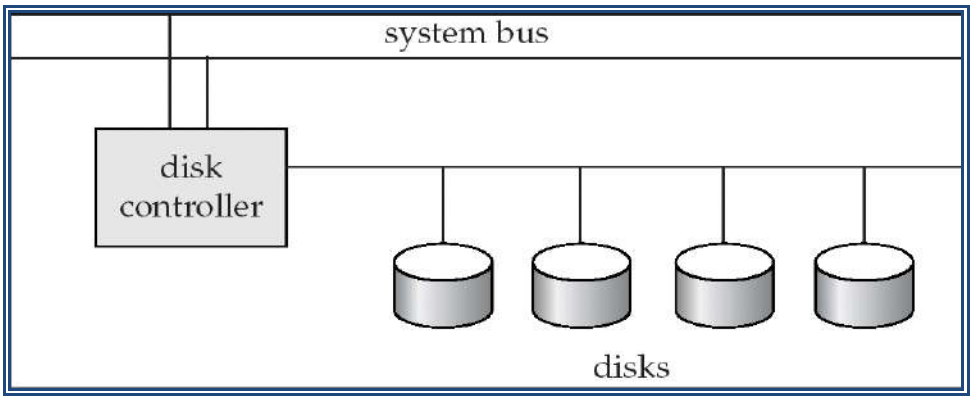
- **Read-write head**
  - ✓ Positioned very close to the platter surface (almost touching it)
  - ✓ Reads or writes magnetically encoded information.
  - ✓ Surface of platter divided into circular **tracks**
  - ✓ Over 50K-100K tracks per platter on typical hard disks
- Each track is divided into **sectors.**
  - ✓ Sector size typically 512 bytes
  - ✓ Typical sectors per track: 500 (on inner tracks) to 1000 (on outer tracks)
- To read/write a sector
  - ✓ disk arm swings to position head on right track
  - ✓ platter spins continually; data is read/written as sector passes under head

- Head-disk assemblies
  - ✓ multiple disk platters on a single spindle (1 to 5 usually)
  - ✓ One head per platter, mounted on a common arm.
- **Cylinder** *c*onsists of *i*th track of all the platters
- Earlier generation disks were susceptible to "head-crashes" leading to loss of all data on disk
  - ✓ Current generation disks are less susceptible to such disastrous failures, but individual sectors may get corrupted

Disk controller – interfaces between the computer system and the disk drive hardware.

- accepts high-level commands to read or write a sector
- initiates actions such as moving the disk arm to the right track and actually reading or writing the data
- Computes and attaches **checksums** to each sector to verify that data is read back correctly
  - If data is corrupted, with very high probability stored checksum won't match recomputed checksum
- Ensures successful writing by reading back sector after writing it
- Performs remapping of bad sectors

Disk Subsystem



2. Describe the following **(April 2012)**

A). Transaction Isolation

B). Serializability

TRANSACTIONS

- Collections of operations that form a single logical unit of work are called as transactions.
- A database system must ensure proper execution of transactions despite failures—either the entire transaction executes, or none of it does.
- A transaction is a unit of program execution that accesses and possibly updates various data items.
- Usually, a transaction is initiated by a user program written in a High-level data-manipulation language or programming language, where it is delimited by statements (or function calls) of the form begin transaction and end transaction.
- To ensure integrity of the data, the database system maintain the following properties of the transactions:
  - **Atomicity**. Either all operations of the transaction are reflected properly in the database, or none.
  - **Consistency**. Execution of a transaction in isolation (that is, with no other transaction executing concurrently) preserves the consistency of the database.
  - **Isolation**. Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions *Ti* and *Tj,* it appears to *Ti* that either *Tj* finished execution before *Ti* started, or *Tj* started execution after *Ti* finished. Thus, each transaction is unaware of other transactions executing concurrently in the system.
  - **Durability**. After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

  Transactions access data using **two operations**:

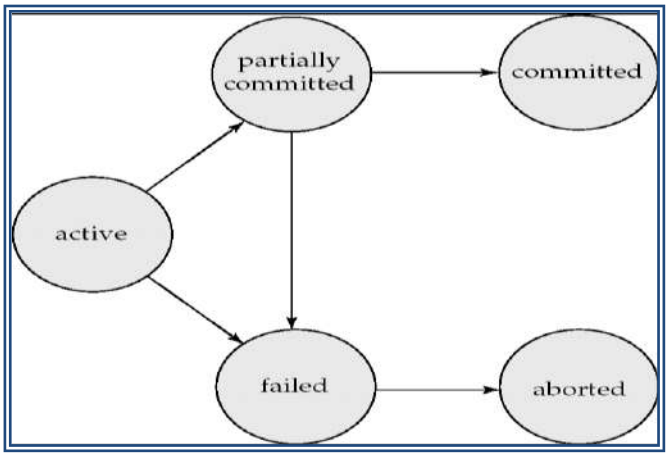
  1. **read(*X*),** which transfers the data item *X* from the database to a local buffer belonging to the transaction that executed the read operation.
  2. **write(*X*),** which transfers the data item *X* from the local buffer of the transaction that executed the write back to the database.

3. Transaction State: (April 2014)

A transaction must be in one of the following states:

1. Active, the initial state; the transaction stays in this state while it is executing.
2. Partially committed, after the final statement has been execute.
3. Failed, after the discovery that normal execution can no longer Proceed.
4. Aborted, after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.
5. Committed, after successful completion.

- A transaction starts in the active state.
- When it finishes its final statement, it enters the partially committed state.
- The database system then writes out enough information to disk.
- If a transaction enters the failed state such a transaction must be rolled back.
- Then, it enters the aborted state. At this point, the system has two options:
  - It can restart the transaction.
  - It can kill the transaction.
- The state diagram corresponding to a transaction appears as follows:



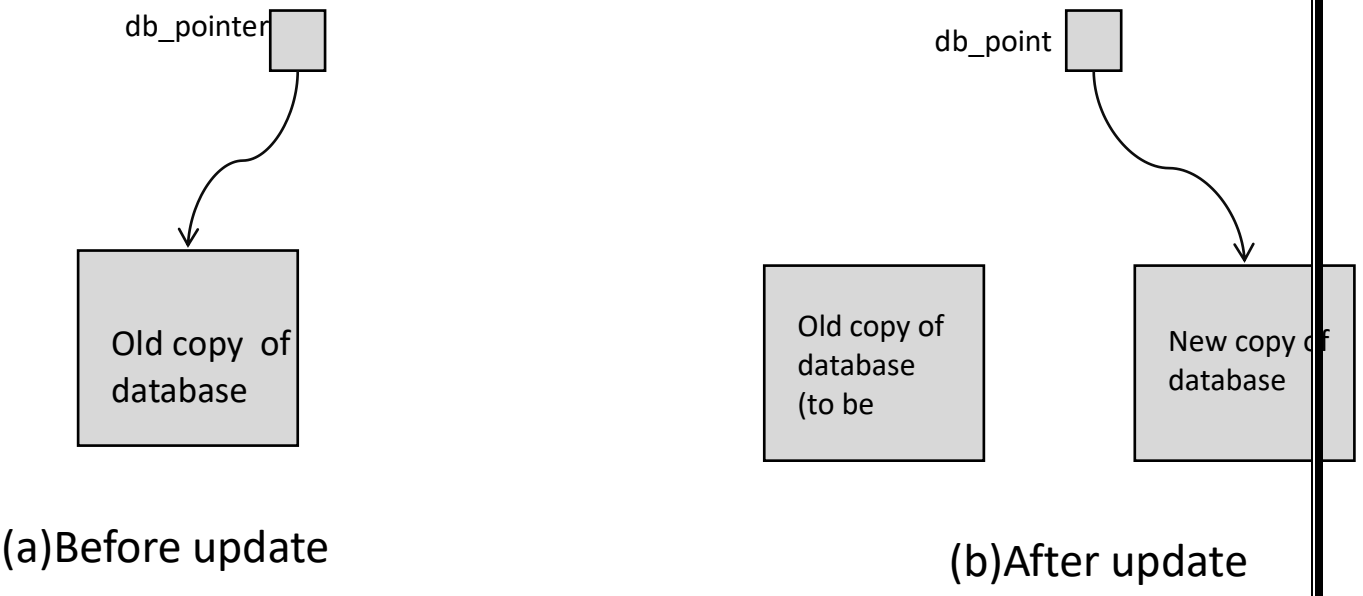
Implementation of Atomicity and Durability:

- Recovery-management component of a database system can support atomicity and durability by a variety of schemes.

Shadow Copy

- It is simple, but extremely inefficient
- It is based on making copies of the database, called *shadow* copies, assumes that only one transaction is active at a time.

- A pointer called db-pointer is maintained on disk; it points to the current copy of the database.
- A transaction that wants to update the database,
  - first creates a complete copy of the database
  - All updates are done on the new database copy,
  - leaving the original copy, the shadow copy, untouched.
  - If at any point the transaction has to be aborted, the system merely deletes the new copy.
  - The old copy of the database has not been affected.



Shadow-copy technique for atomicity and durability.

- The transaction is said to have been *committed* at the point where the updated db-pointer is written to disk.
- If the transaction fails at any time before db-pointer is updated, the old contents of the database are not affected.
- Suppose that the system fails at any time before the updated db-pointer is written to disk.
- Then, when the system restarts, it will read db-pointer and will thus see the original contents of the database, and none of the effects of the transaction will be visible on the database.

- Next, suppose that the system fails after db-pointer has been updated on disk.
- Before the pointer is updated, all updated pages of the new copy of the database were written to disk.
- Thus, the atomicity and durability properties of transactions are ensured by the shadow-copy implementation of the recovery-management component.

Concurrent Executions: (or) Concurrency Control(NOV 2014)(NOV 2015) [DEC 2016](Nov 2018)

- Transaction-processing systems usually allow multiple transactions to run concurrently.
- Allowing multiple transactions to update data concurrently causes several complications with consistency of the data.
- There are two good reasons for allowing concurrency:
- **Improved throughput and resource utilization**. A transaction consists of many steps. Some involve I/O activity; others involve CPU activity. Therefore, I/O activity can be done in parallel with processing at the CPU. All of this increases the throughput of the system correspondingly; the processor and disk utilization also increases.
- **Reduced waiting time.** There may be a mix of transactions running on a system, some short and some long. Concurrent execution reduces the unpredictable delays in running transactions. Moreover, it also reduces the average response time: the average time for a transaction to be completed after it has been submitted.
- The motivation for using concurrent execution in a database is essentially the same as the motivation for using multiprogramming in an operating system.
- Let $T1$ and $T2$ be two transactions that transfer funds from one account to another. Transaction $T1$ transfers \$50 from account A to account B. It is defined as:

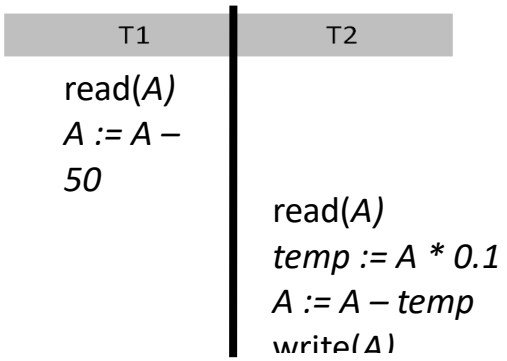
$T1$: read (A);
$A: = A - 50$;
write (A);
read (B);
$B: = B + 50$;
write (B).

- Transaction *T*2 transfers 10 percent of the balance from account *A* to account *B*. It is defined as:

  *T*2: read (*A*);
  temp: =A * 0.1;
  A: =A – temp;
  write (*A*);
  read (*B*);
  B: =B + temp;
  write (*B*).

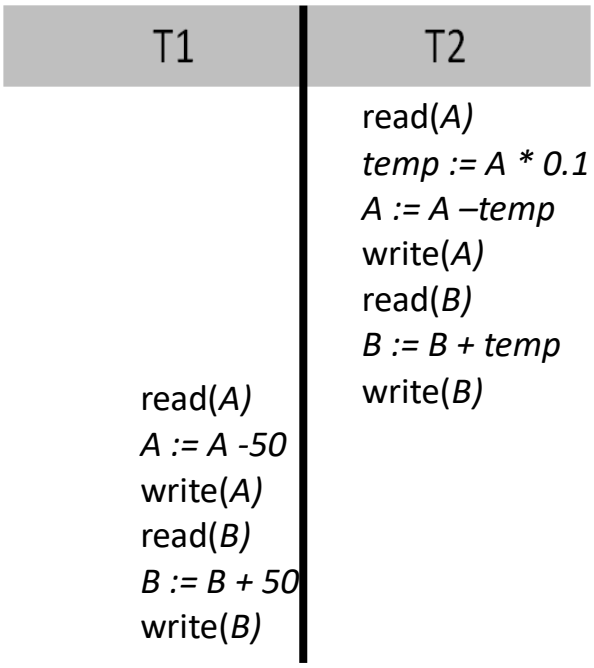
- Suppose the current values of accounts *A* and *B* are $1000 and $2000, respectively.

- Suppose also that the two transactions are executed one at a time in the order *T*1 followed by *T*2. This execution sequence appears as follows:



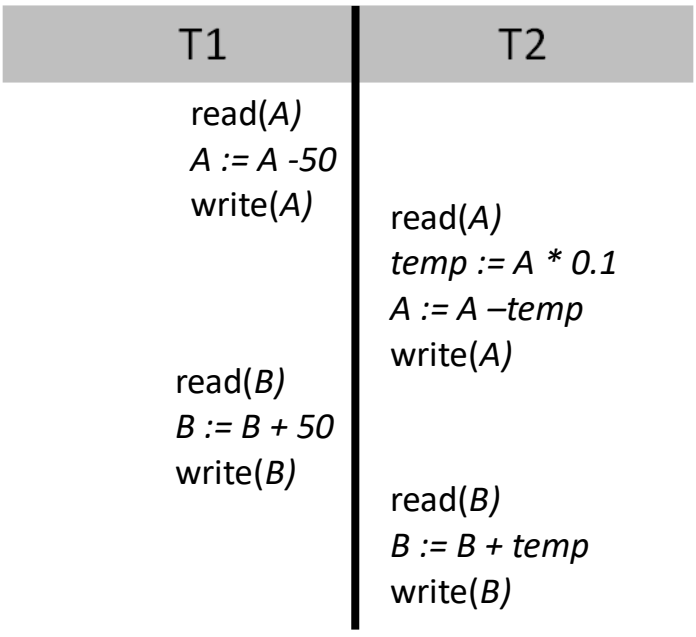
Schedule 1—a serial schedule in which *T*1 is followed by *T*2.

- The final values of accounts *A* and *B*, after the execution in this figure takes place, are $855 and $2145, respectively.
- Similarly, if the transactions are executed one at a time in the order *T*2 followed by *T*1, then the corresponding execution sequence is as follows:



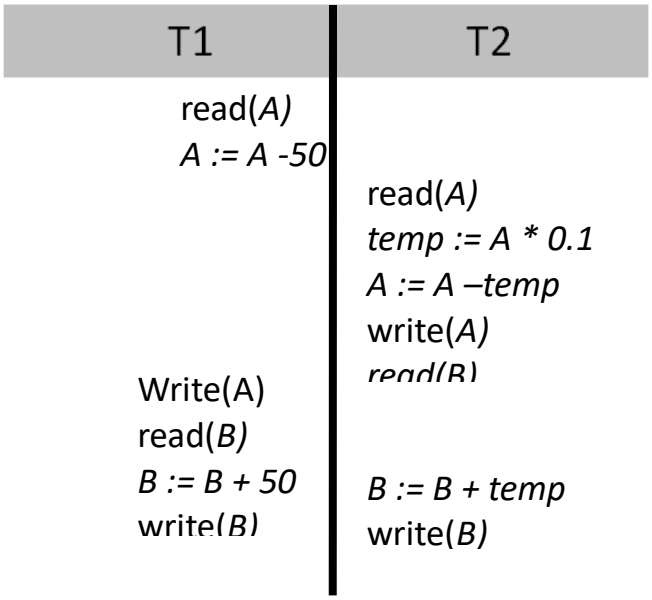
Schedule 2—a serial schedule in which *T*2 is followed by *T*1.

- Again, as expected, the sum $A + B$ is preserved, and the final values of accounts A and B are \$850 and \$2150, respectively.
- The execution sequences just described are called schedules.
- suppose that the two transactions are executed concurrently as follows:



Schedule 3—a concurrent schedule equivalent to schedule 1.

- After this execution takes place, we arrive at the same state as the one in which the transactions are executed serially in the order $T1$ followed by $T2$. The sum $A + B$ is indeed preserved.
- Not all concurrent executions result in a correct state. To illustrate, consider the schedule as follows:



Schedule 4—a concurrent schedule.

- After the execution of this schedule, we arrive at a state where the final values of accounts A and B are \$950 and \$2100, respectively.

- This final state is an *inconsistent state*.

Serializability

The database system must control concurrent execution of transactions, to ensure that the database state remains consistent.

Since transactions are programs, it is computationally difficult to determine exactly what operations a transaction performs and how operations of various transactions interact.

Two operations:

read and write. We thus assume that, between a read(Q) instruction and a write(Q) instruction on a data item Q, a transaction may perform an arbitrary sequence of operations on the copy of Q that is residing in the local buffer of the transaction.

Thus, the only significant operations of a transaction, from a scheduling point of view, are its read and write instructions.

T1	T2
read(A) write(A)	 read(A) write(A)

Schedule3-showing only the read and write instructions

Conflict Serializability

Let us consider a schedule *S* in which there are two consecutive instructions *Ii* and *Ij*, of transactions *Ti* and *Tj* , respectively (*i* _= *j*). If *Ii* and *Ij* refer to different data items, then we can swap *Ii* and *Ij* without affecting the results of any instruction in the schedule.

However, if *Ii* and *Ij* refer to the same data item *Q*, then the order of the two steps may matter. Since we are dealing with only read and write instructions, there are four cases that we need to consider:

1. *Ii* = read(*Q*), *Ij* = read(*Q*). The order of *Ii* and *Ij* does not matter, since the same value of *Q* is read by *Ti* and *Tj* , regardless of the order.
2. *Ii* = read(*Q*), *Ij* = write(*Q*). If *Ii* comes before *Ij*, then *Ti* does not read the value of *Q* that is written by *Tj* in instruction *Ij*. If *Ij* comes before *Ii*, then *Ti* reads the value of *Q* that is written by *Tj*. Thus, the order of *Ii* and *Ij* matters.
3. *Ii* = write(*Q*), *Ij* = read(*Q*). The order of *Ii* and *Ij* matters for reasons similar to those of the previous case.
4. *Ii* = write(*Q*), *Ij* = write(*Q*). Since both instructions are write operations, the order of these instructions does not affect either *Ti* or *Tj* . However, the value obtained by the next read(*Q*) instruction of *S* is affected, since the result of only the latter of the two write instructions is preserved in the database. If there is no other write(*Q*) instruction after *Ii* and *Ij* in *S*, then the order of *Ii* and *Ij* directly affects the final value of *Q* in the database state that results from schedule *S*.

The write(A) instruction of $T1$ conflicts with the read(A) instruction of $T2$.

However, the write(A) instruction of $T2$ does not conflict with the read(B) instruction of $T1$, because the two instructions access different data items.

View Serializability

Consider two schedules S and $S\_$, where the same set of transactions participates in both schedules. The schedules S and $S\_$ are said to be view equivalent if three conditions are met:

1. For each data item Q, if transaction Ti reads the initial value of Q in schedule S, then transaction Ti must, in schedule $S\_$, also read the initial value of Q.

2. For each data item Q, if transaction Ti executes read(Q) in schedule S, and if that value was produced by a write(Q) operation executed by transaction Tj , then the read(Q) operation of transaction Ti must, in schedule $S\_$, also read the value of Q that was produced by the same write(Q) operation of transaction Tj .

3. For each data item Q, the transaction (if any) that performs the final write(Q) operation in schedule S must perform the final write(Q) operation in schedule $S\_$.

T1	T2
read(A)	
A:=A-50	
Write(A)	
	read(B)
	B:=B-10
	write(B)
read(B)	

Schedule 8.

The concept of view equivalence leads to the concept of view serializability. We say that a schedule S is view serializable if it is view equivalent to a serial schedule.

T3	T4	T6
Read(Q) Write(Q)	Write(Q)	

Schedule 9—a view-serializable schedule.

5. Discuss about recovery and Atomicity (or)

Discuss about the Recovery System for Database Management Systems.

(NOV 2014)

Recovery system

Recovering a system from failure crash is called recovery system or crash recovery.

Failure Classification:

Various types of failure that may occur in a system

1. Transaction failure

A. Logical errors - The Transaction cannot complete due to some internal error condition

B.System errors-The database system must terminate an active transaction due to an error condition. (e.g., deadlock)

2. System crash:

A power failure or other hardware or software failure causes the system to crash.

Fail stop assumption

A non-volatile storage contents are assumed to not be corrupted by system crash. Database systems have numerous integrity checks to prevent corruption of disk data

3. Disk failure

A head crash or similar disk failure destroys all or part of Disk. Destruction is assumed to be detectable: disk drives use checksums to detect failures

4. Recovery Algorithms

Recovery algorithms are techniques to ensure database consistency and transaction atomicity and durability despite failures.

Recovery algorithms have two parts

6. Actions taken during normal transaction processing to ensure enough information exists to recover from failures
7. Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability.

Storage Structure :

The various data items in the database may be stored and accessed in a number of different storage media.

Storage types:

The types are:

- Volatile: It does not survive system crashes.

eg: main memory, cache memory

- Non volatile storage: It survives system crashes

eg: disk, tape, flash memory, non-volatile (battery backed up) RAM.

- Stable storage: A mythical form of storage that survives all failures approximated by maintaining multiple copies on distinct nonvolatile media.

Stable Storage Implementation

- Maintain multiple copies of each block on separate disks
  - copies can be at remote sites to protect against disasters such as fire or flooding.
- Failure during data transfer can still result in inconsistent copies: Block transfer can result in
  - Successful completion

    Information arrived safely at its destination
  - Partial failure

    Destination block has incorrect information
  - Total failure:

    Destination block was never updated .
- Protecting storage media from failure during data transfer (one solution)
  - Execute output operation as follows (assuming two copies of each block)

1. Write the information onto the first physical block.
2. When the first write successfully completes, write the Same information onto the second physical block.
3. The output is completed only after the second write successfully completes.

- copies of a block may differ due to failure during output operation. To recover from failure:

First find inconsistent blocks:

1. Expensive solution: Compare the two copies of every disk block.

2. Better solution: Record in-progress disk writes on non- volatile storage (Nonvolatile RAM or special area of disk).

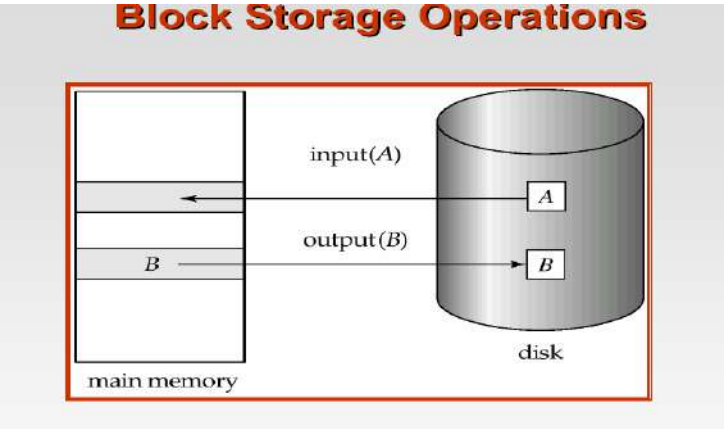
- Use this information during recovery to find blocks that may be inconsistent, and only compare copies of these.
- Used in hardware RAID systems If either copy of an inconsistent block is detected to have an error (bad checksum), overwrite it by the other copy. If both have no error, but are different, overwrite the second block by the first block.

Data Access

- Physical blocks are those blocks residing on the disk.
- Buffer blocks are the blocks residing temporarily in main memory.

Block movements between disk and main memory are initiated through the following two operations:

- input(*B*) transfers the physical block *B* to main memory.
- output(*B*) transfers the buffer block *B* to the disk, and replaces the appropriate physical block there.
- Each transaction *Ti* has its private work-area in which local copies of all data items accessed and updated by it are kept.
- *Ti*'s local copy of a data item *X* is called *xi.*



We assume, for simplicity, that each data item fits in, and is stored inside, a single block.

- Transaction transfers data items between system buffer

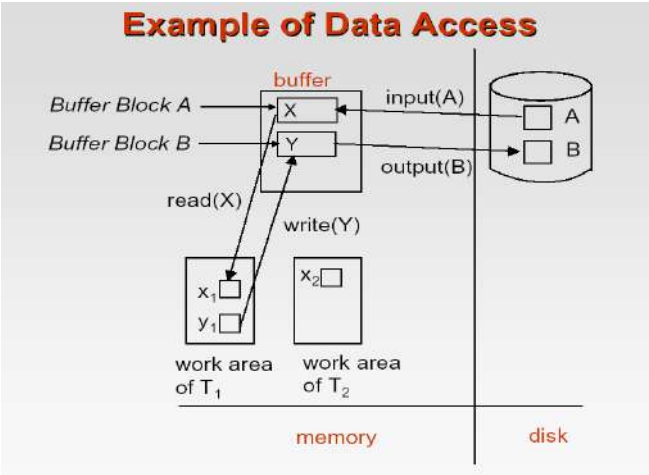
blocks and its private work-area using the following operations :

- read(X) assigns the value of data item X to the local variable xi.
- write(X) assigns the value of local variable xi to data item {X} in the buffer block.
- Both these commands may necessitate the issue of an input(BX) instruction before the assignment, if the block BX in which X resides is not already in memory.

Transactions:

- Perform read(X) while accessing X for the first time;
- All subsequent accesses are to the local copy.After last access, transaction executes write(X).
- output(BX) need not immediately follow write(X). System

can perform the output operation when it deems fit.



Recovery and Atomicity

- Modifying the database without ensuring that the transaction will commit may leave the database in an inconsistent state.
- Consider transaction *Ti* that transfers $50 from account *A* to account *B*;goal is either to perform all database modifications made by *Ti* or none at all.
- Several output operations may be required for *Ti* (to output *A* and *B*). A failure may occur after one of these modifications have been made but before all of them are made.

To ensure atomicity despite failures, we first output information describing the modifications to stable storage without modifying the database itself.

Two approaches:

1. log based recovery
2. Shadow paging

Assume that a transaction run serially, that is, one after the other.

Log Based Recovery

- A log is kept on stable storage.
- The log is a sequence of log records, and maintains a record of update activities on the database.
- When transaction *Ti* starts, it registers itself by writing a<*Ti*

start>log record

- *Before Ti* executes write(*X*), a log record <*Ti, X, V1, V2*>is written,
- where *V1* is the value of *X* before the write, and *V2* is the value to be written to x.
- Log record notes that *Ti* has performed a write on data item

Xj Xj had value *V1* before the write, and will have value *V2* after the write.

- When *Ti* finishes it last statement, the log record <*Ti*

commit> is written to .x

We assume for now that log records are written directly to stable storage (that is, they are not buffered)

  - Two approaches using logs
  - Deferred database modification
  - Immediate database modification

Deferred Database Modification

- The deferred database modification scheme records all modifications to the log, but defers all the writes to after partial commit.
- Assume that transactions execute serially
- Transaction starts by writing <*Ti start*>record to log.
- A write(*X*) operation results in a log record *<Ti, X, V>*being

written, where *V* is the new value for *X.*

✓ Note: old value is not needed for this scheme.

- The write is not performed on *X* at this time, but is deferred. When *Ti* partially commits, <*Ti* commit> is written to the log .
- Finally, the log records are read and used to actually execute the previously deferred writes.
- During recovery after a crash, a transaction needs to be redone if and only if both <*Ti* start> and<*Ti* commit> are there in the log.
- Redoing a transaction *Ti* (redo*Ti*) sets the value of all data items updated by the transaction to the new values.
- Crashes can occur while the transaction is executing the original updates, or while recovery action is being taken

example: transactions *T0* and *T1* (*T0* executes before *T1*):

T0 : read (*A*)

T1 : read (*C*)

A: - A - 50 *C:- C- 100*

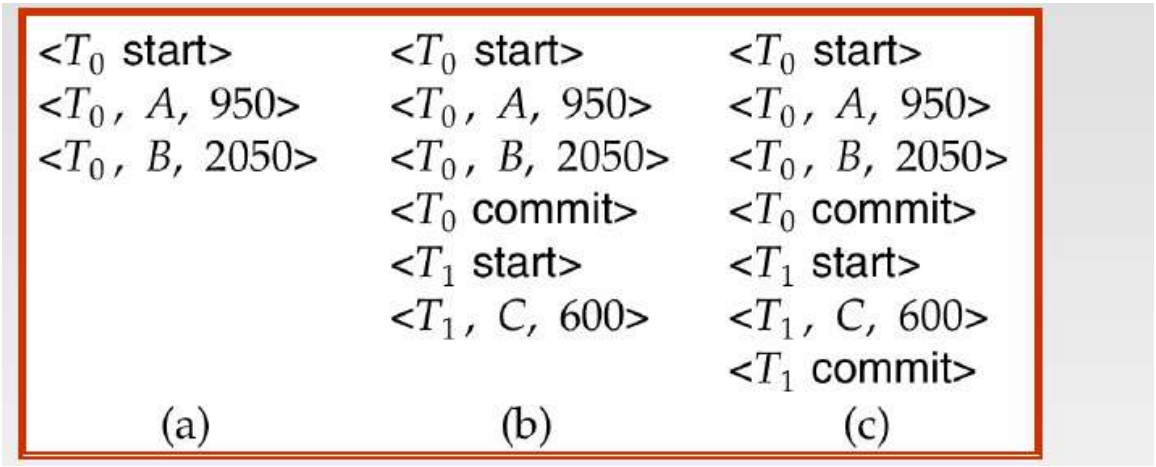
Write (*A*) write (*C*)

read (*B*)

B:- B + 50

write (*B*)

Below we show the log as it appears at three instances of time.



- If log on stable storage at time of crash is as in case:

(a) No redo actions need to be taken

(b) redo($T0$) must be performed since <$T0$ commit> is present

(c) redo($T0$) must be performed followed by redo($T1$) since <$T0$ commit> and <Ti commit> are present

Immediate Database Modification

- The immediate database modification scheme allows database updates of an uncommitted transaction to be made as the writes are issued.
- since undoing may be needed, update logs must have both old value and new value
- Update log record must be written *before* database item is written.
- We assume that the log record is output directly to stable storage
- Can be extended to postpone log record output, so long as prior to execution of an output(B) operation for a data block B, all log records corresponding to items B must be flushed to stable storage.
- Output of updated blocks can take place at any time before or after transaction commit.
- Order in which blocks are output can be different from the order in which they are written.

Example :

Log input output

Log Write Output

<T0 start>

<T0, A, 1000, 950>

To, B, 2000, 2050

A = 950

B = 2050

<T0 commit>

<T1 start>

<T1, C, 700, 600>

C = 600

BB, BC

<T1 commit>

BA

Note: BX denotes block containing X.

x1

Recovery procedure has two operations instead of one:

- undo(Ti) restores the value of all data items updated by Ti

to their old values, going backwards from the last log record for Ti

- redo(Ti) sets the value of all data items updated by Ti to the

new values, going forward from the first log record for Ti.

- Both operations must be idempotent That is, even if the operation is executed multiple times the effect is the same as if it is executed once
- Needed since operations may get re-executed during recovery

When recovering after failure:

- Transaction Ti needs to be undone if the log contains the record
- <Ti start>, but does not contain the record <Ti commit>.
- Transaction Ti needs to be redone if the log contains both the record
- <Ti start> and the record <Ti commit>.

6. Discuss fixed length records in detail with an example

FIXED-LENGTH RECORDS:

For an example, let us consider a file of account records for our bank database. Each record of this file is defined as:

```
type deposit=record
account-number:char(10);
branch-name:char(22);
  balance:real;
  end
```

- If we assume that each character occupies 1 byte and that a real occupies 8 bytes, our account record is 40 bytes long. A simple approach is to use the first 40 bytes for the first record, the next 40 bytes for the second record, and so on.
- Though, we have two problems with this simple approach:

1. It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of this file, or we must have way of marking deleted records so that they can be ignored.
2. Unless the block happens to be a multiple of 40(which is unlikely), some records will cross block boundaries. That is, part of the record will be stored in one block and part in another. It would thus require two block accesses to read or write such a record.
3. When a record is deleted, we could move the record that came after it into the space formerly occupied by the deleted record and so on, until every record following the deleted record has been moved ahead.

FIG 1.2 ,WITH RECORD 2 DELETED AND ALL RECORDS MOVED

Record 0	A-102	Perryridge	400
Record 1	A-305	Round Hill	350
Record 3	A-101	Downtown	500
Record 4	A-222	Redwood	700
Record 5	A-201	Perryridge	900
Record 6	A-217	Brighton	750
Record 7	A-110	Downtown	600
Record 8	A-218	Perryridge	700

- It might be easier simply to move the final record of the file into the space occupied by the deleted record.(fig-1.3)

Fig-1.3 WITH RECORD 2 DELETED AND FINAL RECORD MOVED

Record 0	A-102	Perryridge	400
Record 1	A-305	Round Hill	350
Record 8	A-218	Perryridge	700
Record 3	A-101	Downtown	500
Record 4	A-222	Redwood	700
Record 5	A-201	Perryridge	900
Record 6	A-217	Brighton	750
Record 7	A-110	Downtown	600

- A simple marker on a deleted record is not sufficient, since it is hard to find this available space when an insertion is being done. Thus, we need to introduce an additional structure.
- At the beginning of the file, we allocate a certain number of bytes as a *FILE HEADER.*
- The header will contain a variety of information about the file.
- For now, all we need to store there is the address of the first record whose contents are deleted. We use this first record to store the address of the second available record and so on.
- These deleted records form a linked list, which is often referred to as a *FREE LIST*.

Fig 1.4, *WITH FREE LIST AFTER DELETION OF RECORDS 1,4,AND 6*

Header			
Record 0	A-102	Perryridge	400
Record 1			
Record 2	A-215	Mianus	700
Record 3	A-101	Downtown	500
Record 4			
Record 5	A-201	Perryridge	900
Record 6			
Record 7	A-110	Downtown	600
Record 8	A-218	Perryridge	700

- On insertion of a new record, we use the record pointed to by the header.

- We change the header pointer to point to the next available record. If no space is available, we add the new file to the end of the record.

♦ NOTE: Insertion and deletion for files of fixed-length records are simple to implement, because

♦ The space made available by a deleted record is exactly the space needed to insert a record. An inserted record may not fit in the space left free by a deleted record, or it may fill only part of that space.

7. Explain storage access in detail **(APR 2010)**

Storage Access

A database file is partitioned into fixed length storage units called **blocks**. Blocks are units of both storage allocation and data transfer.

Database system seeks to minimize the number of block transfers between the disk and memory. We can reduce the number of disk accesses by keeping as many blocks as possible in main memory.

Buffer – portion of main memory available to store copies of disk blocks.

Buffer manager – subsystem responsible for allocating buffer space in main memory.

Buffer Manager

Programs call on the buffer manager when they need a block from disk. Buffer manager does the following:

- If the block is already in the buffer, return the address of the block in main memory
    1. . If the block is not in the buffer
        - . Allocate space in the buffer for the block
        - . Replacing (throwing out) some other block, if required, to make space for the new block.
- Replaced block written back to disk only if it was modified since the most recent time that it was written to/fetched from the disk.
- Read the block from the disk to the buffer, and return the address of the block in main memory to requester.

8. What is the transactions isolation level in SQL? How to implementation of isolation level. Discuss it.

Concurrent Executions:

- Transaction-processing systems usually allow multiple transactions to run concurrently.
- Allowing multiple transactions to update data concurrently causes several complications with consistency of the data.
- There are two good reasons for allowing concurrency:
- **Improved throughput and resource utilization**. A transaction consists of many steps. Some involve I/O activity; others involve CPU activity. Therefore, I/O activity can be done in parallel with processing at the CPU. All of this increases the throughput of the system correspondingly; the processor and disk utilization also increases.
- **Reduced waiting time.** There may be a mix of transactions running on a system, some short and some long. Concurrent execution reduces the unpredictable delays in running transactions. Moreover, it also reduces the average response time: the average time for a transaction to be completed after it has been submitted.
- The motivation for using concurrent execution in a database is essentially the same as the motivation for using multiprogramming in an operating system.
- Let $T1$ and $T2$ be two transactions that transfer funds from one account to another. Transaction $T1$ transfers \$50 from account A to account B. It is defined as:

  $T1$: read (A);
  $A: = A - 50$;
  write (A);
  read (B);
  $B: = B + 50$;
  write (B).

- Transaction $T2$ transfers 10 percent of the balance from account A to account B. It is defined as:

  $T2$: read (A);
  $temp: = A * 0.1$;
  $A: = A - temp$;

write (A);

read (B);

B: =B + $temp$;

write (B).

- Suppose the current values of accounts A and B are $1000 and $2000, respectively.
- Suppose also that the two transactions are executed one at a time in the order $T1$ followed by $T2$. This execution sequence appears as follows:

T1	T2
read(A)	
$A := A - 50$	
write (A)	
read(B)	
$B := B + 50$	
write(B)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)

Schedule 1—a serial schedule in which $T1$ is followed by $T2$.

- The final values of accounts A and B, after the execution in this figure takes place, are $855 and $2145, respectively.

- Similarly, if the transactions are executed one at a time in the order $T2$ followed by $T1$, then the corresponding execution sequence is as follows:

T1	T2
	read(*A*)
	*temp := A * 0.1*
	A := A –temp
	write(*A*)
	read(*B*)
	B := B + temp
read(*A*)	write(*B*)
A := A -50	
write(*A*)	
read(*B*)	
B := B + 50	
write(*B*)	

Schedule 2—a serial schedule in which $T2$ is followed by $T1$.

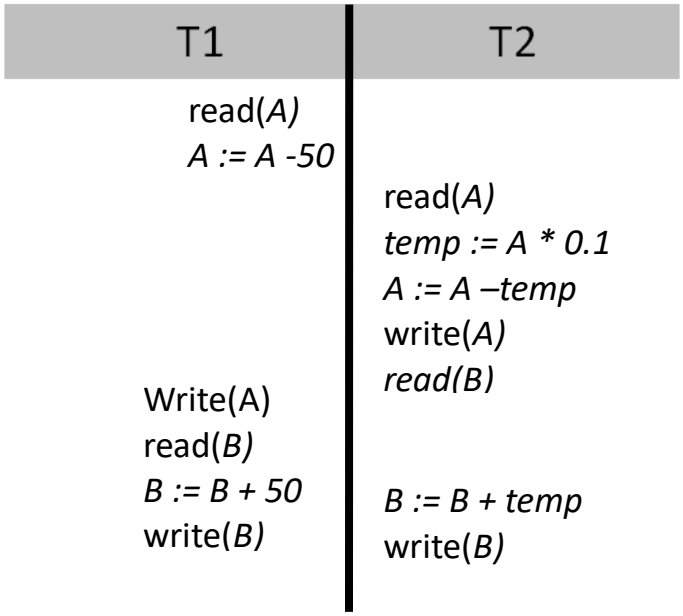
- Again, as expected, the sum $A + B$ is preserved, and the final values of accounts A and B are \$850 and \$2150, respectively.
- The execution sequences just described are called schedules.
- suppose that the two transactions are executed concurrently as follows:

T1	T2
read(*A*)	
A := A -50	
write(*A*)	
	read(*A*)
	*temp := A * 0.1*
	A := A –temp
	write(*A*)
read(*B*)	
B := B + 50	
write(*B*)	
	read(*B*)
	B := B + temp
	write(*B*)

Schedule 3—a concurrent schedule equivalent to schedule 1.

- After this execution takes place, we arrive at the same state as the one in which the transactions are executed serially in the order $T1$ followed by $T2$. The sum $A + B$ is indeed

preserved. Not all concurrent executions result in a correct state. To illustrate, consider the schedule as follows:



Schedule 4—a concurrent schedule.

- After the execution of this schedule, we arrive at a state where the final values of accounts *A* and *B* are $950 and $2100, respectively.

- This final state is an *inconsistent state*.

9. Explain about Transaction control in detail. (April 2015)

TRANSACTIONS (11 Marks)

- Collections of operations that form a single logical unit of work are called as transactions.
- A transaction is a unit of program execution that accesses and possibly updates various data items.
- To ensure integrity of the data, the database system maintain the following properties of the transactions:
  - **Atomicity**. Either all operations of the transaction are reflected properly in the database, or none.
  - **Consistency**. Execution of a transaction in isolation (that is, with no other transaction executing concurrently) preserves the consistency of the database.
  - **Isolation**. Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions *Ti* and *Tj,* it appears to *Ti* that either *Tj* finished execution before *Ti* started, or *Tj* started execution after *Ti* finished. Thus, each transaction is unaware of other transactions executing concurrently in the system.

- **Durability**. After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Transactions access data using **two operations**:

i. **read(*X*),** which transfers the data item *X* from the database to a local buffer belonging to the transaction that executed the read operation.

ii. **write(*X*),** which transfers the data item *X* from the local buffer of the transaction that executed the write back to the database.

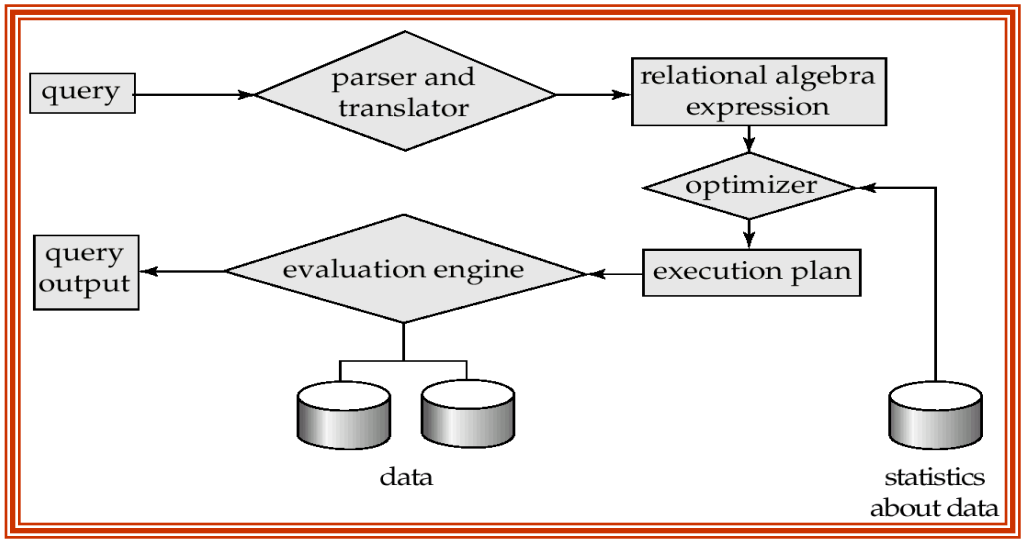
10. **Briefly explain Query processing and Optimization in Oracle.** (**Nov 2013**) (**April 2014**) (**Nov 2014**) (NOV 2015)(DEC 2018) (MAY 2019)

Basic Steps in Query Processing

1 Parsing and translation

2. Optimization

3. Evaluation



Parsing and translation

- translate the query into its internal form. This is then translated into relational algebra.
- Parser checks syntax, verifies relations Evaluation
- The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.

Basic Steps in Query Processing: Optimization

A relational algebra expression may have many equivalent expressions

- E.g., $\sigma_{balance<2500}(\Pi_{balance}(account))$ is equivalent to

  $\Pi_{balance}(\sigma_{balance<2500}(account))$

Each relational algebra operation can be evaluated using one of several different algorithms correspondingly, a relational-algebra expression can be evaluated in many ways. Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**.

- E.g., can use an index on *balance* to find accounts with balance < 2500,
- or can perform complete relation scan and discard accounts with balance ≥ 2500

Query Optimization: Amongst all equivalent evaluation plans choose the one with lowest cost.

- Cost is estimated using statistical information from the database catalog
  - e.g. number of tuples in each relation, size of tuples, etc.

In this chapter we study

- How to measure query costs
- Algorithms for evaluating relational algebra operations
- How to combine algorithms for individual operations in order to evaluate a complete expression

Measures of Query Cost

Cost is generally measured as total elapsed time for answering query

- Many factors contribute to time cost
  - *disk accesses, CPU*, or even network *communication*

Typically disk access is the predominant cost, and is also relatively easy to estimate. Measured by taking into account

- Number of seeks * average-seek-cost
- Number of blocks read * average-block-read-cost
- Number of blocks written * average-block-write-cost
  - Cost to write a block is greater than cost to read a block
    - data is read back after being written to ensure that the write was successful

For simplicity we just use *number of block transfers from disk* as the cost measure

- We ignore the difference in cost between sequential and random I/O for simplicity
- We also ignore CPU costs for simplicity

Costs depends on the size of the buffer in main memory

- Having more memory reduces need for disk access
- Amount of real memory available to buffer depends on other concurrent OS processes, and hard to determine ahead of actual execution

★ We often use worst case estimates, assuming only the minimum amount of memory needed for the operation is available

11.Explain the transactions properties in detail with examples

TRANSACTIONS

- Collections of operations that form a single logical unit of work are called as transactions.
- A database system must ensure proper execution of transactions despite failures—either the entire transaction executes, or none of it does.
- A transaction is a unit of program execution that accesses and possibly updates various data items.
- Usually, a transaction is initiated by a user program written in a High-level data-manipulation language or programming language, where it is delimited by statements (or function calls) of the form begin transaction and end transaction.
- To ensure integrity of the data, the database system maintain the following properties of the transactions:

  - Atomicity. Either all operations of the transaction are reflected properly in the database, or none are.
  - Consistency. Execution of a transaction in isolation (that is, with no other transaction executing concurrently) preserves the consistency of the database.
  - Isolation. Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions *Ti* and *Tj,* it appears to *Ti* that either *Tj* finished execution before *Ti* started, or *Tj* started execution after *Ti* finished. Thus, each transaction is unaware of other transactions executing concurrently in the system.
  - Durability. After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Properties of Transactions

ATOMICITY

all or nothing

CONSISTENCY

no violation of integrity constraints

ISOLATION

concurrent changes invisible Þserializable

DURABILITY

committed updates persist

- These properties are often called the ACID properties; the acronym is derived from the first letter of each of the four properties.

- Transactions access data using two operations:

1. **read(*X*),** which transfers the data item *X* from the database to a local buffer belonging to the transaction that executed the read operation.
2. **write(*X*)**, which transfers the data item *X* from the local buffer of the transaction that executed the write back to the database.

- Let *Ti* be a transaction that transfers $50 from account *A* to account *B*. This transaction can be defined as

```
Ti: read (A);
A: = A − 50;
write (A);
read (B);
B: = B + 50;
write (B).
```

• Consistency: The consistency requirement here is that the sum of *A* and *B* be unchanged by the execution of the transaction.

• Atomicity: Suppose that, just before the execution of transaction *Ti* the values of accounts *A* and *B* are $1000 and $2000, respectively. Suppose that the failure happened after the write (*A*) operation but before the write (*B*) operation. In this case, the values of accounts *A* and *B* reflected in the

database are $950 and $2000. The system destroyed $50 as a result of this failure. Such a state is termed as inconsistent state. If the atomicity property is present, all actions of the transaction are reflected in the database, or none are.

• Durability: The durability property guarantees that, once a transaction completes successfully, all the updates that it carried out on the database persist, even if there is a system failure after the transaction completes execution. We can guarantee durability by ensuring that either

- The updates carried out by the transaction have been written to disk before the transaction completes.
- Information about the updates carried out by the transaction and written to disk is sufficient to enable the database to reconstruct the update when the database system is restarted after the failure.

• Isolation: Even if the consistency and atomicity properties are ensured for each transaction, if several transactions are executed concurrently, their operations may interleave in some undesirable way, resulting in an inconsistent state. A way to avoid the problem of concurrently executing transactions is to execute transactions serially—that is, one after the other. Ensuring the isolation property is the responsibility of a component of the database system called the concurrency-control component executes transactions serially—that is, one after the other.

12. Consider the following relational schemes for a library database: [April 2017]

```
Book (Title, Author, Catalog_no, Publisher, Year, Price)
Collection(Title, Author, Catalog_no)
```

with the following functional dependencies:

I. Title Author → Catalog_noTitle Author → Catalog_no
II. Catalog_no → Title Author Publisher YearCatalog_no → Title Author Publisher Year
III. Publisher Title Year→ PricePublisher Title Year→ Price

Assume { Author, Title }{ Author, Title } is the key for both schemes. Which of the following statements is true?

A. Both Book and Collection are in BCNF
B. Both Book and Collection are in 3NF only
C. Book is in 2NF and Collection in 3NF
D. Both Book and Collection are in 2NF only

Answer: C

It is given that *{Author , Title}* is the key for both schemas.

The given dependencies are :

- {Title, Author} --> {Catalog_no}
- Catalog_no --> {Title , Author , Publisher , Year }
- {Publisher , Title , Year} --> {Price}

First , let's take schema *Collection (Title , Author , Catalog_no) :*

{Title , Author} --> Catalog_no

{Title, Author} is a candidate key and hence super key also and by definition of BCNF this is in BCNF.

Now , let's see *Book (Title , Author , Catalog_no , Publisher , Year , Price) :*

{Title , Author}+ --> {Title , Author , Catalog_no , Publisher , Year , Price}

{Catalog_no}+--> {Title , Author , Publisher , Year , Price , Catalog_no}

So candidate keys are : Catalog_no , {Title , Author}

But in the given dependencies , {Publisher , Title , Year} --> Price , which has Transitive Dependency. **So , Book is in 2NF.**

13. Explain Nested Join, Hash Join, and Merge Join in SQL Query Plan.[April 2017]

Nested Loops Joins

- Nested loops join an outer data set to an inner data set.
- For each row in the outer data set that matches the single-table predicates, the database retrieves all rows in the inner data set that satisfy the join predicate.
- If an index is available, then the database can use it to access the inner data set by rowid.

Hash Joins

- The database uses a hash join to join larger data sets.
- The optimizer uses the smaller of two data sets to build a hash table on the join key in memory, using a deterministic hash function to specify the location in the hash table in which to store each row.
- The database then scans the larger data set, probing the hash table to find the rows that meet the join condition.

Sort Merge Joins

- A sort merge join is a variation on a nested loops join.
- The database sorts two data sets (the SORT JOIN operations), if they are not already sorted.

- For each row in the first data set, the database probes the second data set for matching rows and joins them (the MERGE JOIN operation), basing its start position on the match made in the previous iteration.

UNIVERSITY QUESTIONS

1. Explain typically available storages media in detail **(April 2011)[Question No.: 01]**

2. Describe the following **(April 2012) (April 2014)[Question No.: 02]**

   A). Transaction Isolation & Transaction State

   B). Serializability , Conflict Serializability **(NOV 2015)**

3. Discuss about recovery and Atomicity **(April 2012) [Question No.: 05]**

4. Discuss fixed length records in detail with an example. **(April 2013)[Question No.: 06]**

5. Explain storage access in detail **(Nov 2010) [Question No.: 07]**

6. What is the transactions isolation level in SQL? How to implementation of isolation level.

   Discuss it**. [Question No.: 08]**

7. Explain about the Concurrency Control component of the database system can ensure serializability. **(NOV 2014)(April 2018)[Question No.: 04]** **[DEC 2016]**

8. Explain about the Recovery System for Database Management System. **(NOV 2014)**

   [Question No.: 05]

9. Explain about Transaction control in detail**. (APRIL 2015) [Question No.: 09]**

10. Briefly explain Query processing and Optimization in Oracle. (**Nov 2013**) (**April 2014**) (**Nov 2014**) (NOV 2015) (Nov 2017)**[Question No.: 10]** **[DEC 2016] (DEC2018)(MAY 2019)**

11. Consider the following relational schemes for a library database: [April 2017]

[Question No.: 12]

```
Book (Title, Author, Catalog_no, Publisher, Year, Price)
Collection(Title, Author, Catalog_no)
```

with the following functional dependencies:

IV. Title Author → Catalog_noTitle Author → Catalog_no
V. Catalog_no → Title Author Publisher YearCatalog_no → Title Author Publisher Year
VI. Publisher Title Year→ PricePublisher Title Year→ Price

Assume { Author, Title }{ Author, Title } is the key for both schemes. Which of the following statements is true?

E. Both Book and Collection are in BCNF
F. Both Book and Collection are in 3NF only
G. Book is in 2NF and Collection in 3NF

H. Both Book and Collection are in 2NF only

13. Explain Nested Join, Hash Join, and Merge Join in SQL Query Plan.[April 2017][Question No:13]

14 .Explain sort-merge algorithm and compute the cost analysis algorithm?(April 2018)